

Zomato

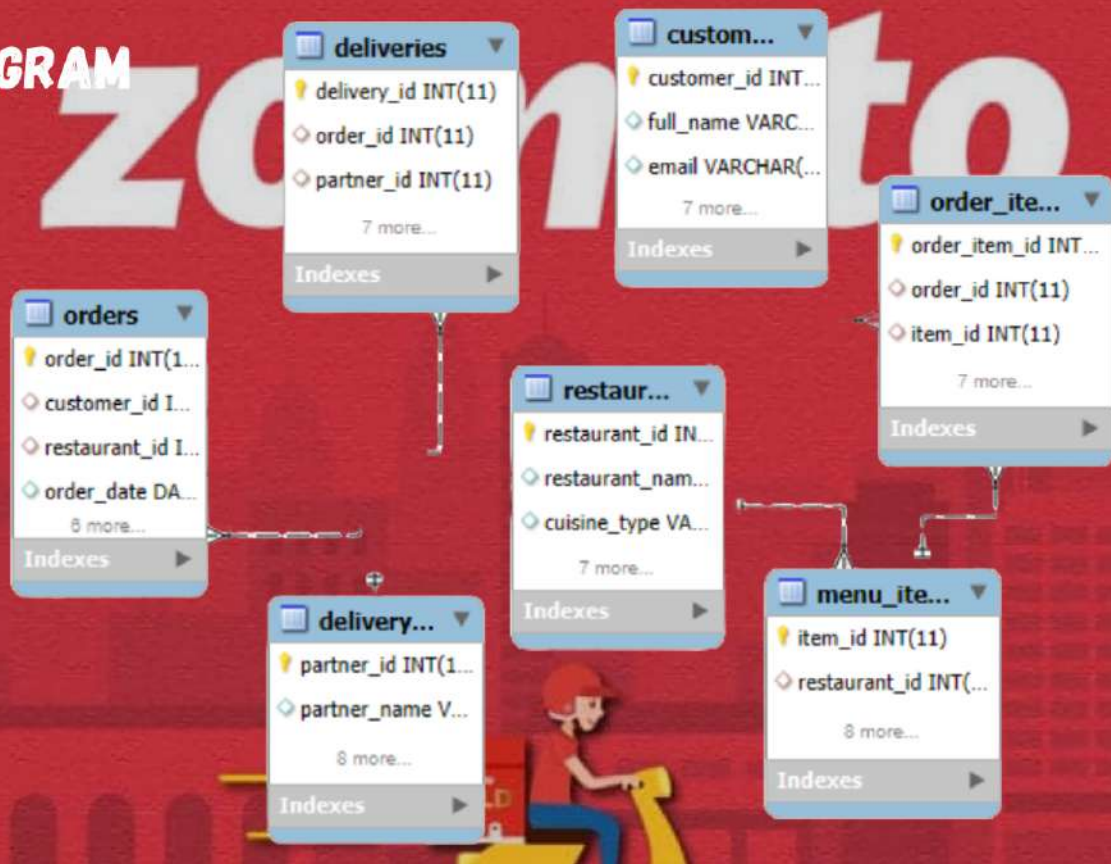
DEVELOPED BY : SHREYAS WARANG



ABSTRACT

ZOMATO IS A LEADING ONLINE FOOD DELIVERY AND RESTAURANT DISCOVERY PLATFORM THAT CONNECTS USERS WITH RESTAURANTS THROUGH REAL-TIME ORDERING, TRACKING, AND REVIEW SYSTEMS. THIS PROJECT MODELS THE CORE OPERATIONS OF ZOMATO BY DESIGNING A FULLY STRUCTURED AND OPTIMIZED DATABASE THAT CAPTURES INFORMATION ABOUT USERS, RESTAURANTS, MENUS, DISHES, ORDERS, DELIVERY PARTNERS, PAYMENTS, RATINGS, AND CUSTOMER FEEDBACK. THE SYSTEM SUPPORTS ESSENTIAL PLATFORM FUNCTIONALITIES SUCH AS ORDER PLACEMENT, MENU BROWSING, DELIVERY ASSIGNMENT, TRANSACTION PROCESSING, AND PERFORMANCE ANALYTICS. A SERIES OF SQL QUERIES—COVERING JOINS, AGGREGATION, SUBQUERIES, AND WINDOW FUNCTIONS—DEMONSTRATES HOW THE DATABASE ENABLES EFFICIENT DATA RETRIEVAL AND SUPPORTS DECISION-MAKING PROCESSES LIKE TRENDING DISHES, TOP-RATED RESTAURANTS, CUSTOMER BEHAVIOR ANALYSIS, AND DELIVERY PERFORMANCE TRACKING. THIS PROJECT HIGHLIGHTS THE IMPORTANCE OF A ROBUST RELATIONAL DATABASE IN POWERING SCALABLE, RELIABLE, AND REAL-TIME FOOD DELIVERY PLATFORMS LIKE ZOMATO.

E-R DIAGRAM



zomato

STRUCTURE OF TABLE

TABLE CUSTOMERS

- `describe Customers;`

Result Grid

Filter Rows:

Export:

Wrap

	Field	Type	Null	Key	Default	Extra
▶	customer_id	int(11)	NO	PRI	NULL	
	full_name	varchar(100)	YES		NULL	
	email	varchar(100)	YES		NULL	
	phone	varchar(15)	YES		NULL	
	gender	varchar(10)	YES		NULL	
	city	varchar(50)	YES		NULL	
	dob	date	YES		NULL	
	created_at	date	YES		NULL	
	membership	varchar(20)	YES		NULL	
	referral_code	varchar(20)	YES		NULL	

THE TABLE CONTAINS BASIC CUSTOMER DETAILS USED FOR IDENTIFICATION AND CONTACT. IT INCLUDES FIELDS SUCH AS EMAIL_ , PHONE_ , GENDER_ , CITY_ , DOB_ , CREATE_AT_ , MEMBERSHIP_ , REFERRAL_CODE_ .

zomato

STRUCTURE OF TABLE

TABLE RESTAURANTS

```
describe Restaurants;
```

	Field	Type	Null	Key	Default	Extra
►	restaurant_id	int(11)	NO	PRI	NULL	
	restaurant_name	varchar(100)	YES		NULL	
	cuisine_type	varchar(50)	YES		NULL	
	rating	decimal(3,1)	YES		NULL	
	city	varchar(50)	YES		NULL	
	delivery_time	int(11)	YES		NULL	
	opening_time	time	YES		NULL	
	closing_time	time	YES		NULL	
	is_active	tinyint(1)	YES		NULL	
	created_at	date	YES		NULL	

THIS TABLE STORES INFORMATION ABOUT RESTAURANTS . IT INCLUDE FIELDS SUCH AS RESTAURANT_ID_ , RESTAURANT_NAME_ ,CUISINE_TYPE_ ,RATING_ ,CITY_ , DELIVERY_TIME_ , OPENING_TIME_ , CLOSING_TIME_ , IS_ACTIVE_,CREATED_AT_ .

zomato

STRUCTURE OF TABLE

TABLE MENU_ITEMS

```
describe Menu_Items;
```

Result Grid		Filter Rows:	Export:		Wrap	
	Field	Type	Null	Key	Default	Extra
▶	item_id	int(11)	NO	PRI	NULL	
	restaurant_id	int(11)	YES	MUL	NULL	
	item_name	varchar(100)	YES		NULL	
	category	varchar(50)	YES		NULL	
	price	decimal(7,2)	YES		NULL	
	is_veg	tinyint(1)	YES		NULL	
	spice_level	varchar(20)	YES		NULL	
	calories	int(11)	YES		NULL	
	available	tinyint(1)	YES		NULL	
	created_at	date	YES		NULL	

THIS TABLE STORES INFORMATION ABOUT MENU_ITEMS. IT INCLUDE FIELDS SUCH AS ITEMS_ID_, RESTAURANT_ID_, ITEM_NAME_, CATEGORY_, PRICE_, IS_VEG_, SPICE_LEVEL_, CALORIES_, AVAIABLE_, CREATED_AT_.

zomato

STRUCTURE OF TABLE

TABLE ORDERS

```
describe Orders;
```

Field	Type	Null	Key	Default	Extra
order_id	int(11)	NO	PRI	NULL	
customer_id	int(11)	YES	MUL	NULL	
restaurant_id	int(11)	YES	MUL	NULL	
order_date	datetime	YES		NULL	
payment_method	varchar(20)	YES		NULL	
order_status	varchar(20)	YES		NULL	
total_amount	decimal(10,2)	YES		NULL	
discount	decimal(7,2)	YES		NULL	
final_amount	decimal(10,2)	YES		NULL	
delivery_address	varchar(150)	YES		NULL	

THIS TABLE STORES INFORMATION ABOUT ORDERS. IT INCLUDE FIELDS SUCH AS ORDERS_ID_,CUSTOMER_ID_, RESTAURANT_ID_,ORDER_DATE_,PAYMENT_METHOD_,ORDER_STATUS, TOTAL_AMOUNT_, DISCOUNT_, FINAL_AMOUNT_,DELIVERY_ADDRESS_.

zomato

STRUCTURE OF TABLE

TABLE ORDER_ITEMS

```
describe Order_Items;
```

Field	Type	Null	Key	Default	Extra
order_item_id	int(11)	NO	PRI	NULL	
order_id	int(11)	YES	MUL	NULL	
item_id	int(11)	YES	MUL	NULL	
quantity	int(11)	YES		NULL	
price	decimal(10,2)	YES		NULL	
subtotal	decimal(10,2)	YES		NULL	
taste_preference	varchar(20)	YES		NULL	
spice_preference	varchar(20)	YES		NULL	
instructions	varchar(100)	YES		NULL	
added_at	datetime	YES		NULL	

THIS TABLE STORES INFORMATION ABOUT ORDER_ITEMS. IT INCLUDE FIELDS SUCH AS ORDER_ITEMS_ID,ORDER_ID, ITEM_ID,QUANTITY,PRICE,SUBTOTAL, TASTE_PREFERENCE, SPICE_PREFERENCE, INSTRUCTIONS,ADDED_AT.



STRUCTURE OF TABLE

TABLE DELIVERY_PARTNERS

```
describe Delivery_Partners;
```

Field	Type	Null	Key	Default	Extra
partner_id	int(11)	NO	PRI	HULL	
partner_name	varchar(100)	YES		HULL	
phone	varchar(15)	YES		HULL	
vehide_type	varchar(50)	YES		HULL	
city	varchar(50)	YES		HULL	
rating	decimal(3,1)	YES		HULL	
total_deliveries	int(11)	YES		HULL	
cancelled_deliveries	int(11)	YES		HULL	
joining_date	date	YES		HULL	
active	tinyint(1)	YES		HULL	

THIS TABLE STORES INFORMATION ABOUT DELIVERY_PARTNERS. IT INCLUDE FIELDS SUCH AS PARTNER_ID_,PARTNER_NAME_, PHONE_,VEHICLE_TYPE_,CITY_,RATING_, TOTAL_DELIVERIES_, CANCELLED_DELIVERIES_, JOINING_DATE_,ACTIVE_.

zomato

STRUCTURE OF TABLE

TABLE DELIVERIES

```
describe Deliveries;
```

Result Grid						
		Filter Rows:			Export:	Wrap C
	Field	Type	Null	Key	Default	Extra
►	delivery_id	int(11)	NO	PRI	NULL	
	order_id	int(11)	YES	MUL	NULL	
	partner_id	int(11)	YES	MUL	NULL	
	pickup_time	datetime	YES		NULL	
	drop_time	datetime	YES		NULL	
	delivery_status	varchar(20)	YES		NULL	
	distance_km	decimal(5,2)	YES		NULL	
	delivery_charge	decimal(7,2)	YES		NULL	
	tip	decimal(7,2)	YES		NULL	
	created_at	date	YES		NULL	

THIS TABLE STORES INFORMATION ABOUT DELIVERIES. IT INCLUDE FIELDS SUCH AS DELIVERIES_ID, ORDER_ID, PARTNER_ID, PICKUP_TIME, DROP_TIME, DELIVERY_STATUS, DISTANCE_KM, DELIVERIES_CHARGE, TRIP, CREATED_AT.

[illegible]



CONTENTS OF TABLE

```
select * from Restaurants;
```

[illegible]



CONTENTS OF TABLE

```
select * from Menu_Items;
```

[illegible]

[illegible]

[illegible]

CONTENTS OF TABLE

```
select * from Delivery_Partners;
```

[illegible]

CONTENTS OF TABLE

```
select * from Deliveries;
```

[illegible]

zomato

JOINS

A JOIN IS USED TO COMBINE ROWS FROM TWO OR MORE TABLES BASED ON A RELATED COLUMN, ALLOWING YOU TO RETRIEVE DATA FROM MULTIPLE TABLES IN A SINGLE QUERY



LIST ALL ORDERS WITH CUSTOMER NAMES AND RESTAURANT NAMES.

```
SELECT o.order_id, c.full_name, r.restaurant_name, o.final_amount
FROM Orders o
JOIN Customers c ON o.customer_id = c.customer_id
JOIN Restaurants r ON o.restaurant_id = r.restaurant_id;
```

CONCLUSION:

THIS QUERY SUCCESSFULLY COMBINES CUSTOMER, ORDER, AND RESTAURANT DATA USING JOINS TO PRESENT A COMPLETE VIEW OF EACH ORDER. IT DEMONSTRATES HOW RELATIONAL TABLES WORK TOGETHER TO GENERATE MEANINGFUL, BUSINESS-READY INSIGHTS.

Result Grid

  Filter Rows:

Export: 

	order_id	full_name	restaurant_name	final_amount
▶	1	Amit Sharma	Pizza Hut	450.00
	2	Riya Kapoor	Burger King	280.00
	3	Suresh Kumar	KFC	400.00
	4	Priya Singh	Behrouz Biryani	540.00
	5	Rahul Mehta	Subway	240.00
	6	Neha Jain	Dominos	420.00
	7	Karan Patel	Barbeque Nation	850.00
	8	Simran Kaur	Taco Bell	330.00
	9	Deepak Verma	Haldirams	200.00
	10	Anjali Mishra	Wow Momo	400.00



SHOW MENU ITEMS ORDERED ALONG WITH THE RESTAURANT NAME.

```
SELECT oi.order_item_id, mi.item_name, r.restaurant_name, oi.quantity
FROM Order_Items oi
JOIN Menu_Items mi ON oi.item_id = mi.item_id
JOIN Restaurants r ON mi.restaurant_id = r.restaurant_id;
```

CONCLUSION:

THIS QUERY LINKS ORDER ITEMS WITH MENU AND RESTAURANT DETAILS TO SHOW WHAT ITEMS WERE ORDERED FROM EACH RESTAURANT. IT HIGHLIGHTS HOW JOINS HELP TRACK ITEM-LEVEL SALES ACROSS DIFFERENT RESTAURANTS.

Result Grid   Filter Rows: Export: 				
	order_item_id	item_name	restaurant_name	quantity
▶	1	Farmhouse Pizza	Pizza Hut	1
	2	Whopper Burger	Burger King	1
	3	Zinger Burger	KFC	2
	4	Chicken Biryani	Behrouz Biryani	1
	5	Veg Sub	Subway	2
	6	Paneer Pizza	Dominos	1
	7	Grilled Chicken	Barbeque Nation	1
	8	Taco Supreme	Taco Bell	2
	9	Rajma Chawal	Haldirams	1
	10	Veggie Pizza	Pizza Hut	1



GET ALL DELIVERIES WITH PARTNER NAMES AND CUSTOMER CITY.

```
SELECT d.delivery_id, dp.partner_name, c.city, d.delivery_status
FROM Deliveries d
JOIN Delivery_Partners dp ON d.partner_id = dp.partner_id
JOIN Orders o ON d.order_id = o.order_id
JOIN Customers c ON o.customer_id = c.customer_id;
```

CONCLUSION:

THIS QUERY INTEGRATES DELIVERY, PARTNER, ORDER, AND CUSTOMER DATA TO PROVIDE A COMPLETE VIEW OF DELIVERY STATUS BY CITY. IT DEMONSTRATES HOW JOINS ENABLE TRACKING DELIVERY PERFORMANCE AND CANCELLATIONS ACROSS REGIONS.

Result Grid

 Filter Rows:

Export:

	delivery_id	partner_name	city	delivery_status
▶	1	Rohit Kumar	Mumbai	Delivered
	2	Aman Singh	Delhi	Delivered
	3	Vikas Yadav	Bangalore	Cancelled
	4	Neeraj Gupta	Hyderabad	Delivered
	5	Mohit Sharma	Mumbai	Delivered
	6	Rahul Nair	Pune	Delivered
	7	Sanjay Verma	Ahmedabad	Delivered
	8	Ravi Choudhary	Chandigarh	Delivered
	9	Aditya Das	Jaipur	Delivered
	10	Kunal Patil	Kolkata	Cancelled



FIND ORDERS WITH TOTAL ITEM COUNT PER ORDER.

```
SELECT o.order_id, c.full_name, SUM(oi.quantity) AS total_items
FROM Orders o
JOIN Customers c ON o.customer_id = c.customer_id
JOIN Order_Items oi ON o.order_id = oi.order_id
GROUP BY o.order_id;
```

CONCLUSION:

THIS QUERY CALCULATES THE TOTAL NUMBER OF ITEMS ORDERED IN EACH ORDER BY COMBINING CUSTOMER AND ORDER ITEM DATA. IT DEMONSTRATES THE USE OF AGGREGATION WITH GROUP BY TO SUMMARIZE ORDER-LEVEL DETAILS.

Result Grid				Filter Rows:
	order_id	full_name	total_items	
▶	1	Amit Sharma	1	
	2	Riya Kapoor	1	
	3	Suresh Kumar	2	
	4	Priya Singh	1	
	5	Rahul Mehta	2	
	6	Neha Jain	1	
	7	Karan Patel	1	
	8	Simran Kaur	2	
	9	Deepak Verma	1	
	10	Anjali Mishra	1	



LIST RESTAURANTS WITH NUMBER OF ITEMS IN THEIR MENU.

```
SELECT r.restaurant_name, COUNT(mi.item_id) AS total_items
FROM Restaurants r
LEFT JOIN Menu_Items mi ON r.restaurant_id = mi.restaurant_id
GROUP BY r.restaurant_id;
```

CONCLUSION:

THIS QUERY USES A LEFT JOIN WITH AGGREGATION TO DISPLAY THE NUMBER OF MENU ITEMS AVAILABLE FOR EACH RESTAURANT, INCLUDING THOSE WITH NONE. IT ENSURES COMPLETE RESTAURANT COVERAGE WHILE SUMMARIZING MENU AVAILABILITY.

Result Grid			Filter Rows:
	restaurant_name	total_items	
▶	Pizza Hut	2	
	Burger King	1	
	KFC	1	
	Behrouz Biryani	1	
	Subway	1	
	Dominos	1	
	Barbeque Nation	1	
	Taco Bell	1	
	Haldirams	1	
	Wow Momo	0	

zomato

AGGREGATION & GROUP-BY

- 1. THE AGGREGATION SUMMARIZES DATA USING FUNCTIONS LIKE SUM, COUNT, AVG, MAX, MIN.**
- 2. THE GROUP-BY GROUPS ROWS BASED ON ONE OR MORE COLUMNS SO AGGREGATIONS CAN BE APPLIED TO EACH GROUP.**



COUNT HOW MANY CUSTOMERS ARE IN EACH CITY.

```
SELECT city, COUNT(customer_id) AS total_customers
FROM Customers
GROUP BY city;
```

CONCLUSION:

THIS QUERY GROUPS CUSTOMERS BY CITY TO SHOW THE DISTRIBUTION OF USERS ACROSS LOCATIONS. IT HELPS IDENTIFY CITIES WITH HIGHER CUSTOMER CONCENTRATION FOR TARGETED BUSINESS STRATEGIES.

Result Grid |   Filter Rows:

	city	total_customers
▶	Ahmedabad	1
	Bangalore	1
	Chandigarh	1
	Delhi	1
	Hyderabad	1
	Jaipur	1
	Kolkata	1
	Mumbai	2
	Pune	1



FIND AVERAGE RESTAURANT RATING CITY-WISE.

```
SELECT city, AVG(rating) AS avg_rating
FROM Restaurants
GROUP BY city;
```

CONCLUSION:

THIS QUERY CALCULATES THE AVERAGE RESTAURANT RATING FOR EACH CITY USING AGGREGATION AND GROUPING. IT HELPS COMPARE OVERALL RESTAURANT QUALITY ACROSS DIFFERENT LOCATIONS.

Result Grid			Filter Rows
	city	avg_rating	
▶	Bangalore	4.10000	
	Chennai	4.20000	
	Delhi	4.25000	
	Hyderabad	4.40000	
	Kolkata	4.10000	
	Mumbai	4.25000	
	Nagpur	4.30000	
	Pune	4.00000	



TOTAL REVENUE GENERATED BY EACH RESTAURANT.

```
SELECT r.restaurant_name, SUM(o.final_amount) AS total_revenue
FROM Orders o
JOIN Restaurants r ON o.restaurant_id = r.restaurant_id
WHERE o.order_status = 'Delivered'
GROUP BY r.restaurant_id;
```

CONCLUSION:

THIS QUERY CALCULATES THE TOTAL REVENUE GENERATED BY EACH RESTAURANT FROM DELIVERED ORDERS ONLY. IT HELPS IDENTIFY TOP-PERFORMING RESTAURANTS BASED ON ACTUAL COMPLETED SALES.

Result Grid			Filter Rows:
	restaurant_name	total_revenue	
▶	Pizza Hut	450.00	
	Burger King	280.00	
	Behrouz Biryani	540.00	
	Subway	240.00	
	Dominos	420.00	
	Barbeque Nation	850.00	
	Taco Bell	330.00	
	Haldirams	200.00	



COUNT ORDERS BY PAYMENT METHOD.

```
SELECT payment_method, COUNT(order_id) AS total_orders  
FROM Orders  
GROUP BY payment_method;
```

CONCLUSION:

THIS QUERY SUMMARIZES THE NUMBER OF ORDERS PLACED USING EACH PAYMENT METHOD. IT HELPS ANALYZE CUSTOMER PAYMENT PREFERENCES ACROSS CARD, CASH, AND UPI TRANSACTIONS.

Result Grid			Filter Rows:
	payment_method	total_orders	
▶	Card	4	
	Cash	2	
	UPI	4	



HIGHEST, LOWEST, AND AVERAGE MENU PRICE FOR EACH CATEGORY.

```
SELECT category,  
       MAX(price) AS max_price,  
       MIN(price) AS min_price,  
       AVG(price) AS avg_price  
FROM Menu_Items  
GROUP BY category;
```

CONCLUSION:

THIS QUERY ANALYZES MENU PRICING BY CATEGORY, SHOWING THE MAXIMUM, MINIMUM, AND AVERAGE PRICES. IT HELPS COMPARE COST VARIATIONS ACROSS DIFFERENT FOOD CATEGORIES FOR BETTER PRICING INSIGHTS.

Result Grid   Filter Rows:				
	category	max_price	min_price	avg_price
▶	BBQ	450.00	450.00	450.000000
	Biryani	250.00	250.00	250.000000
	Burger	220.00	199.00	209.500000
	Indian	180.00	180.00	180.000000
	Pizza	350.00	300.00	326.666667
	Sandwich	180.00	180.00	180.000000
	Taco	150.00	150.00	150.000000

zomato

SUBQUERY

A SUBQUERY IS A QUERY NESTED INSIDE ANOTHER QUERY, USED TO RETURN A RESULT THAT IS THEN USED BY THE OUTER QUERY, OFTEN IN A WHERE, HAVING, OR FROM CLAUSE



FIND CUSTOMERS WHO SPENT MORE THAN THE AVERAGE ORDER AMOUNT.

```
SELECT full_name, customer_id
FROM Customers
WHERE customer_id IN (
    SELECT customer_id
    FROM Orders
    GROUP BY customer_id
    HAVING SUM(final_amount) > (SELECT AVG(final_amount) FROM Orders));
```

CONCLUSION:

THIS QUERY IDENTIFIES CUSTOMERS WHOSE TOTAL SPENDING IS HIGHER THAN THE AVERAGE ORDER AMOUNT USING A SUBQUERY. IT HELPS RECOGNIZE HIGH-VALUE CUSTOMERS FOR TARGETED OFFERS AND LOYALTY PROGRAMS.

Result Grid			Filter Rows:
	full_name	customer_id	
▶	Amit Sharma	1	
	Priya Singh	4	
	Neha Jain	6	
	Karan Patel	7	
✱	NULL	NULL	



LIST RESTAURANTS WITH RATING ABOVE THE OVERALL AVERAGE RATING.

```
SELECT restaurant_name, rating
FROM Restaurants
WHERE rating > (SELECT AVG(rating) FROM Restaurants);
```

CONCLUSION:

THIS QUERY RETRIEVES RESTAURANTS WHOSE RATINGS ARE ABOVE THE OVERALL AVERAGE RATING USING A SUBQUERY. IT HELPS IDENTIFY TOP-PERFORMING RESTAURANTS BASED ON CUSTOMER FEEDBACK.

Result Grid			Filter Rows:
	restaurant_name	rating	
▶	Behrouz Biryani	4.4	
	Subway	4.3	
	Barbeque Nation	4.5	
	Haldirams	4.3	



GET THE MOST EXPENSIVE ITEM FROM EVERY RESTAURANT.

```
SELECT item_name, price, restaurant_id
FROM Menu_Items
WHERE price = (
    SELECT MAX(price)
    FROM Menu_Items mi
    WHERE mi.restaurant_id = Menu_Items.restaurant_id);
```

CONCLUSION:

THE QUERY CORRECTLY RETRIEVES THE MOST EXPENSIVE MENU ITEM FOR EACH RESTAURANT BY USING A CORRELATED SUBQUERY. AS SHOWN IN THE RESULT, EACH RESTAURANT IS REPRESENTED BY ITS HIGHEST-PRICED ITEM ONLY.

Result Grid   Filter Rows:			
	item_name	price	restaurant_id
▶	Farmhouse Pizza	350.00	1
	Whopper Burger	199.00	2
	Zinger Burger	220.00	3
	Chicken Biryani	250.00	4
	Veg Sub	180.00	5
	Paneer Pizza	330.00	6
	Grilled Chicken	450.00	7
	Taco Supreme	150.00	8
	Rajma Chawal	180.00	9



FIND CUSTOMERS WHO PLACED MORE ORDERS THAN THE AVERAGE NUMBER OF ORDERS.

```
SELECT customer_id, full_name
FROM Customers
WHERE customer_id IN (
    SELECT customer_id
    FROM Orders
    GROUP BY customer_id
    HAVING COUNT(order_id) >
        (SELECT AVG(cnt) FROM (SELECT COUNT(order_id) AS cnt FROM Orders GROUP BY customer_id) x));
```

The screenshot shows a 'Result Grid' interface. It has a header row with two columns: 'customer_id' and 'full_name'. Below the header, there is a single data row where both columns contain the value 'NULL'. To the left of the data row is a small icon of a star. Above the grid, there are icons for 'Result Grid' (a yellow grid), a blue double-headed arrow, and the text 'Filter Rows'.

	customer_id	full_name
★	NULL	NULL

CONCLUSION:

THE QUERY ATTEMPTS TO FIND CUSTOMERS WITH MORE ORDERS THAN THE AVERAGE, BUT THE RESULT SHOWS NO MATCHING RECORDS. THIS INDICATES THAT NO CUSTOMER HAS AN ORDER COUNT EXCEEDING THE OVERALL AVERAGE IN THE DATASET.



FIND DELIVERY PARTNERS WHO COMPLETED MORE DELIVERIES THAN PARTNER AVERAGE.

```
SELECT partner_name, total_deliveries
FROM Delivery_Partners
WHERE total_deliveries > (
    SELECT AVG(total_deliveries) FROM Delivery_Partners);
```

CONCLUSION:

**THE QUERY IDENTIFIES DELIVERY PARTNERS WHOSE
TOTAL DELIVERIES ARE ABOVE THE AVERAGE.
THE RESULT HIGHLIGHTS THE TOP-PERFORMING
PARTNERS WITH HIGHER-THAN-AVERAGE DELIVERY
COUNTS.**

Result Grid			Filter Rows:
	partner_name	total_deliveries	
▶	Rohit Kumar	500	
	Vikas Yadav	450	
	Mohit Sharma	350	

zomato

WINDOW FUNCTIONS

Window functions in SQL perform calculations across a set of rows related to the current row, without collapsing the result set. They are essential for advanced analytics like ranking, running totals, and moving averages.



RANK RESTAURANTS BY RATING (HIGHEST → LOWEST).

```
SELECT restaurant_name, rating,  
       RANK() OVER (ORDER BY rating DESC) AS rating_rank  
FROM Restaurants;
```

CONCLUSION:

THIS QUERY USES A WINDOW FUNCTION TO RANK RESTAURANTS BASED ON THEIR RATINGS IN DESCENDING ORDER. IT ALLOWS EASY COMPARISON OF RESTAURANT PERFORMANCE WHILE HANDLING TIES APPROPRIATELY.

Result Grid				Filter Rows:
	restaurant_name	rating	rating_rank	
▶	Barbeque Nation	4.5	1	
	Behrouz Biryani	4.4	2	
	Haldirams	4.3	3	
	Subway	4.3	3	
	Pizza Hut	4.2	5	
	Taco Bell	4.2	5	
	Wow Momo	4.1	7	
	KFC	4.1	7	
	Burger King	4.0	9	
	Dominos	4.0	9	



FIND RUNNING TOTAL OF REVENUE BY DATE.

```
SELECT order_date, final_amount,  
       SUM(final_amount) OVER (ORDER BY order_date) AS running_total  
FROM Orders;
```

CONCLUSION:

THIS QUERY CALCULATES A RUNNING TOTAL OF REVENUE OVER TIME USING A WINDOW FUNCTION. IT HELPS TRACK CUMULATIVE SALES TRENDS AND MONITOR BUSINESS GROWTH ACROSS DATES.

Result Grid				Filter Rows:	Ex
	order_date	final_amount	running_total		
▶	2023-02-01 13:10:00	450.00	450.00		
	2023-02-02 14:20:00	280.00	730.00		
	2023-02-03 15:30:00	400.00	1130.00		
	2023-02-04 12:40:00	540.00	1670.00		
	2023-02-05 18:10:00	240.00	1910.00		
	2023-02-06 20:10:00	420.00	2330.00		
	2023-02-07 11:30:00	850.00	3180.00		
	2023-02-08 21:00:00	330.00	3510.00		
	2023-02-09 19:20:00	200.00	3710.00		
	2023-02-10 22:10:00	400.00	4110.00		



SHOW CUSTOMER TOTAL SPENDING AND RANK CUSTOMERS BY SPENDING.

```
SELECT c.full_name,  
       SUM(o.final_amount) AS total_spent,  
       RANK() OVER (ORDER BY SUM(o.final_amount) DESC) AS spending_rank  
FROM Customers c  
JOIN Orders o ON c.customer_id = o.customer_id  
GROUP BY c.customer_id;
```

CONCLUSION:

THIS QUERY CALCULATES EACH CUSTOMER'S TOTAL SPENDING AND RANKS THEM USING A WINDOW FUNCTION. IT HELPS IDENTIFY TOP-SPENDING CUSTOMERS FOR LOYALTY PROGRAMS AND TARGETED PROMOTIONS.

Result Grid   Filter Rows:			
	full_name	total_spent	spending_rank
▶	Amit Sharma	450.00	3
	Riya Kapoor	280.00	8
	Suresh Kumar	400.00	5
	Priya Singh	540.00	2
	Rahul Mehta	240.00	9
	Neha Jain	420.00	4
	Karan Patel	850.00	1
	Simran Kaur	330.00	7
	Deepak Verma	200.00	10
	Anjali Mishra	400.00	5



SHOW EACH DELIVERY PARTNER WITH PERCENTAGE OF CANCELLED DELIVERIES.

```
SELECT partner_name,
       total_deliveries,
       cancelled_deliveries,
       (cancelled_deliveries * 100.0 / total_deliveries)
         AS cancel_percentage,
       NTILE(4) OVER (ORDER BY (cancelled_deliveries * 100.0 / total_deliveries))
         AS performance_quartile
FROM Delivery_Partners;
```

CONCLUSION:
THIS QUERY ANALYZES DELIVERY PARTNER PERFORMANCE BY CALCULATING CANCELLATION PERCENTAGES AND GROUPING PARTNERS INTO PERFORMANCE QUANTILES. IT HELPS IDENTIFY HIGH-PERFORMING AND LOW-PERFORMING PARTNERS FOR OPERATIONAL IMPROVEMENTS.

Result Grid   Filter Rows:						Export: 	Wrap Cell Content: 
	partner_name	total_deliveries	cancelled_deliveries	cancel_percentage	performance_quartile		
▶	Vikas Yadav	450	8	1.77778	1		
	Aditya Das	260	5	1.92308	1		
	Rohit Kumar	500	10	2.00000	1		
	Kunal Patil	220	6	2.72727	2		
	Mohit Sharma	350	12	3.42857	2		
	Ravi Choudhary	280	10	3.57143	2		
	Aman Singh	300	15	5.00000	3		
	Sanjay Verma	300	18	6.00000	3		
	Neeraj Gupta	200	20	10.00000	4		
	Rahul Nair	250	25	10.00000	4		



FIND THE MOST ORDERED ITEM WITH RANK.

```
SELECT mi.item_name,  
       SUM(oi.quantity) AS total_ordered,  
       RANK() OVER (ORDER BY SUM(oi.quantity) DESC) AS popularity_rank  
FROM Order_Items oi  
JOIN Menu_Items mi ON oi.item_id = mi.item_id  
GROUP BY mi.item_name;
```

CONCLUSION:

THIS QUERY CALCULATES THE TOTAL QUANTITY ORDERED FOR EACH MENU ITEM AND RANKS THEM BY POPULARITY USING A WINDOW FUNCTION. IT HELPS IDENTIFY THE MOST AND LEAST POPULAR ITEMS BASED ON CUSTOMER DEMAND.

Result Grid				Filter Rows:
	item_name	total_ordered	popularity_rank	
▶	Chicken Biryani	1	4	
	Farmhouse Pizza	1	4	
	Grilled Chicken	1	4	
	Paneer Pizza	1	4	
	Rajma Chawal	1	4	
	Taco Supreme	2	1	
	Veg Sub	2	1	
	Veggie Pizza	1	4	
	Whopper Burger	1	4	
	Zinger Burger	2	1	

zomato FINAL CONCLUSION

THIS PROJECT SUCCESSFULLY DEMONSTRATES THE DESIGN AND IMPLEMENTATION OF A COMPREHENSIVE FOOD DELIVERY DATABASE SYSTEM MODELED AFTER ZOMATO. THROUGH WELL-STRUCTURED RELATIONAL TABLES, DEFINED CONSTRAINTS, AND A REALISTIC DATASET, THE SYSTEM EFFECTIVELY REPRESENTS REAL-WORLD OPERATIONS SUCH AS RESTAURANT ONBOARDING, MENU MANAGEMENT, ORDER PLACEMENT, PAYMENT PROCESSING, DELIVERY ASSIGNMENT, AND CUSTOMER ENGAGEMENT.

THE SQL QUERIES IMPLEMENTED IN THIS PROJECT HIGHLIGHT BOTH OPERATIONAL AND ANALYTICAL CAPABILITIES, PROVIDING VALUABLE INSIGHTS INTO CUSTOMER BEHAVIOR, RESTAURANT PERFORMANCE, DELIVERY EFFICIENCY, ITEM POPULARITY, AND REVENUE TRENDS. OVERALL, THE PROJECT SHOWCASES STRONG DATABASE DESIGN PRINCIPLES AND PRACTICAL SQL PROFICIENCY, MAKING IT SUITABLE FOR ACADEMIC EVALUATION, PORTFOLIO PRESENTATION, AND GAINING AN INDUSTRY-LEVEL UNDERSTANDING OF LARGE-SCALE FOOD DELIVERY PLATFORMS.

zomato

ZOMATO DATA ANALYTICS

THANK YOU

**“BEHIND EVERY ZOMATO ORDER, THERE’S A DATA STORY SHAPING THE DINING
EXPERIENCE.**

EVERY SEARCH, MENU VIEW, ORDER, AND DELIVERY ADDS A NEW CHAPTER.”